Homework

Homework

- In this homework, we will return to the Todo List app to implement a sorting data structure feature with React.
- Login to your codesandbox and pull up the Todo List app version 4, which is the last time we added code to the app. **Fork and rename as TodoList version 5**.
- Let's look at the steps we will be working on:
  - Add local states in App.js needed for the sort process.
  - Pass the states received in the Nav as props to the All Todo List via the route.
  - Create buttons in AllTodos.js to sort data
  - Destructure props and add onclick events and handler calling to each buttons.
  - Pass the handler calling through Nav component (Nav.js).
  - Create the handler in App.js, sort the list based on parameters received and update the state.

Homework

Step 1:

- Add local states. Open **App.js** and add these additional states:

```
this.state = {
  sortType: "asc",
  listNum: "",
  TodoLists: TodoData
};
```

- Next, pass these states to the Nav component calling:

```
render() {
  return (
    <Nav
      lists1={this.state.TodoLists.TodoList1}
      lists2={this.state.TodoLists.TodoList2}
      sortType={this.state.sortType}
      listNum={this.state.listNum}
    />
  );
}
```

Homework

Step 2:

- Open **Nav.js** and pass the states received as props to the All Todo List component calling in the route:

```
<Route path="/allLists">
<AllTodos
    lists1={props.lists1}
    lists2={props.lists2}
    sortType={props.sortType}
    listNum={props.listNum}
  />
</Route>
```

Homework

Step 3:

- Open **AllTodos.js** and add two buttons to Lists1:

```
  {props.lists1.map((list1) => {
    return <div>{list1.text}</div>;
  })}
  <button
    className="mx-1 mt-2 bg-info text-white border-0"
  >
    Rush
  </button>
  <button
    className="mx-1 mt-2 bg-info text-white border-0"
  >
    Relax
  </button>
</div>
```

Homework

Step 3:

- Next add two buttons to Lists2:

```
    {props.lists2.map((list2) => {
      return <div>{list2.text}</div>;
    })}
    <button
      className="mx-1 mt-2 bg-info text-white border-0"
    >
      Rush
    </button>
    <button
      className="mx-1 mt-2 bg-info text-white border-0"
    >
      Relax
    </button>
  </div>
```

Homework

Step 4:

- This next step is to prepare the todo lists to be passed from the buttons to the handlers – since each todo list will have its own set of sort buttons, we have to ensure we are passing the right list from the button. This first step basically is to destructure the props, (not all props), though we can. We choose only these two lists as they are the only ones we need to pass back to the parent component. The other props as a result don't need to be destructured:

```
export default function AllTodos(props) {
  const { lists1, lists2 } = props;
  return (
```

Homework

Step 4:

- Since we have destructured the lists props, we need to remove props:

```
{props.lists1.map((list1) => {
  return <div>{list1.text}</div>;
})}
```

```
{lists1.map((list1) => {
  return <div>{list1.text}</div>;
})}
```

```
{props.lists2.map((list2) => {
  return <div>{list2.text}</div>;
})}
```

```
{lists2.map((list2) => {
  return <div>{list2.text}</div>;
})}
```

Homework

Step 4:

- Next, add an onClick event to the buttons. We'll start with Lists 1. The event will fire the handler (onSort) that will also pass two arguments to it – one the list and the other the sort type.

```
<button
  className="mx-1 mt-2 bg-info text-white border-0"
  onClick={() => props.onSort(lists1, "asc")}
>
  Rush
</button>
<button
  className="mx-1 mt-2 bg-info text-white border-0"
  onClick={() => props.onSort(lists1, "desc")}
>
  Relax
</button>
```

Homework

Step 4:

- Next, add an onClick event to the buttons for Lists 2.

```
<button
  className="mx-1 mt-2 bg-info text-white border-0"
  onClick={() => props.onSort(lists2, "asc")}
>
  Rush
</button>
<button
  className="mx-1 mt-2 bg-info text-white border-0"
  onClick={() => props.onSort(lists2, "desc")}
>
  Relax
</button>
```

Homework

Step 5:

- With the handler calling from the buttons, we will now add the calling in the Nav component (Nav.js):

```
<AllTodos
    lists1={props.lists1}
    lists2={props.lists2}
    sortType={props.sortType}
    listNum={props.listNum}
    onSort={props.onSort}
  />
</Route>
```

Homework

Step 6:

- Return to App.js, there will be two things need to be done here:
  1. Pass the handler in the calling of the Nav component.
  2. Create the handler that will sort the data and update the states with the new data sorted.
- Let's do the 1st task - pass the handler in the calling of the Nav:

```
<Nav
  lists1={this.state.TodoLists.TodoList1}
  lists2={this.state.TodoLists.TodoList2}
  sortType={this.state.sortType}
  listNum={this.state.listNum}
  onSort={this.onSort}
/>
```

Homework

Step 6:

- For the 2nd task - create the handler above the render method. We will use the arrow function for the handler so a separate binding is not needed. It will receive the two arguments from the buttons as parameters. These parameters are named as such to match the property names in the state:

```
onSort = (listNum, sortType) => {

}

render() {
```

Homework

Step 6:

- Within the handler, we'll use the sort method to sort the list that received as a data parameter. The sort method will sort each element in the list array with current element represented as **a** and the next element as **b**.

```
onSort = (listNum, sortType) => {
  listNum.sort((a, b) => {


  })
}
```

Homework

Step 6:

- The function will return 1st a variable that either contains 1 or -1. 1 being ascending order and the opposite for descending order (-1). This is done using a ternary operator to evaluate which sort type it is received as the parameter.

```
listNum.sort((a, b) => {
  const isReversed = sortType === "asc" ? 1 : -1;
})
```

Homework

Step 6:

- The next script takes the value 1 or -1 to determine how the text elements in the list will be compared. If 1 (ascending), the smallest alphabet value (ex. y is less than z, x is less than y and so on) will be moved to the 1st position, the 2nd least value will be moved to 2nd position an so on. If -1 (descending), the largest alphabet value till be moved to 1st position, and so on. So in that process, a (current alphabet) will always be used to compare to b (next alphabet) and thus continue until the entire list has been compared. The JS method **localeCompare()** is the best method to use for sorting alphabets. When this is done, the arrow function will return the sorted list in ascending or descending order:

```
listNum.sort((a, b) => {
  const isReversed = sortType === "asc" ? 1 : -1;
  return isReversed * a.text.localeCompare(b.text);
})
```

Homework

Step 6:

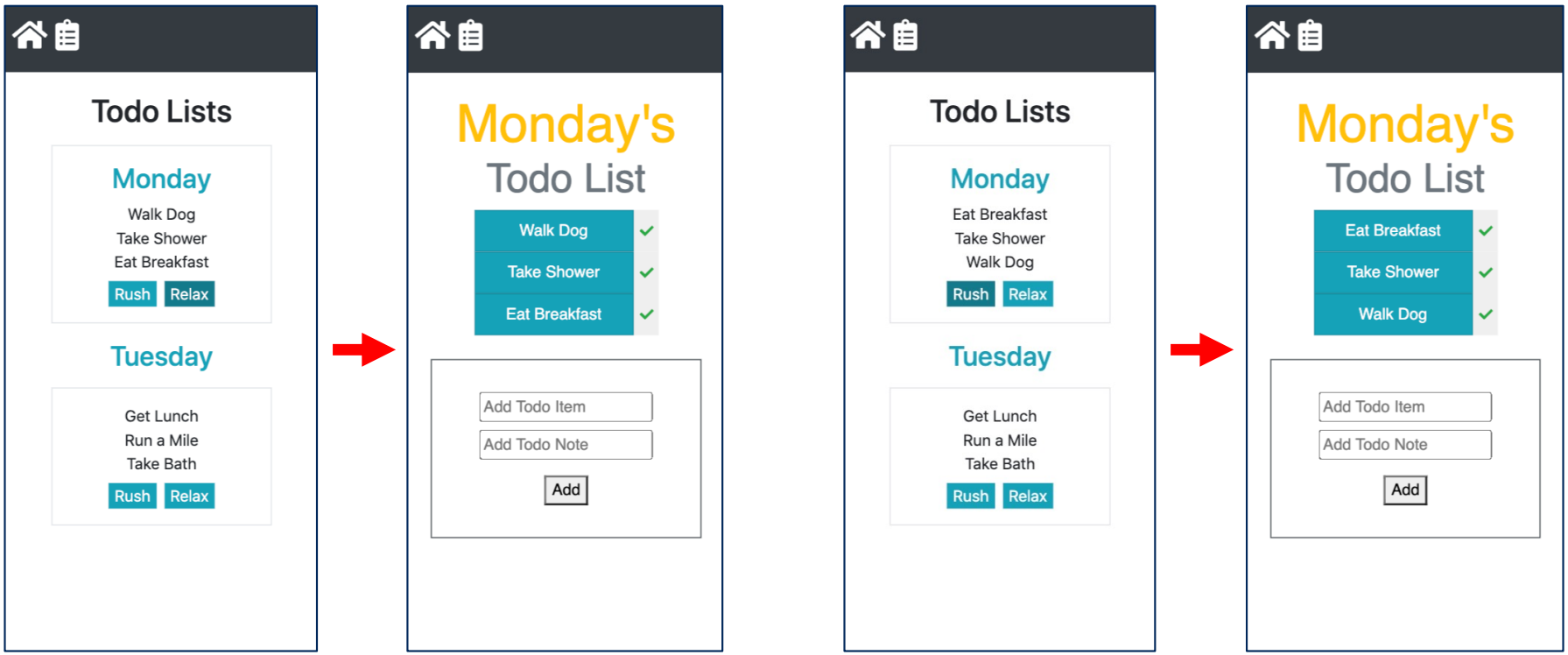
- The final step is to update sort type in the state:

```
onSort = (listNum, sortType) => {
  listNum.sort((a, b) => {
    const isReversed = sortType === "asc" ? 1 : -1;
    return isReversed * a.text.localeCompare(b.text);
  })
  this.setState({ sortType });
}
```

- Our work is complete. Time to give it a spin.

Homework

- Here's what to expect:



Homework

- **DUE:**

  This Sunday, 10.30PM PT

- **Submission:**

  Post Codesandbox link (TodoList V5) on GAP Week 4 Day 2 Homework

Connect with Us (WEB403/603/803)

Rich Loke

richloke@westcliff.edu

Elizabeth Kipp

elizabethkipp@westcliff.edu

Erin Tustin

erintustin@westcliff.edu

End of Presentation